

Pipelined MIPS Simulation: A plug-in to MARS simulator for supporting pipeline simulation and branch prediction

Di Xiang Lim, Smitha K G
School of Computer Science and Engineering
Nanyang Technological University, 50 Nanyang Avenue, Singapore 639798

Email: limd0024@e.ntu.edu.sg, smitha@ntu.edu.sg

Abstract—This paper presents the design and implementation of a Microprocessor without Interlocked Pipeline Stages (MIPS) pipelined simulator build on top of the MIPS Assembler and Runtime Simulator (MARS) as a plug-in. The MARS Simulator is a lightweight interactive development environment (IDE) for programming in MIPS assembly language, intended for education-level use. This plug-in called “**Pipelined Simulator and Branch Explainer (PSBE)**” provides the MARS users with the ability to simulate a pipelined processor execution of MIPS assembly instruction which the original MARS simulator does not support. The tool also provides additional functionality as such Branch Prediction. With the use of the plug-in, MARS users can better analyze the assembly codes executed in a pipelined processor with branch prediction and improve their understanding of Computer Architecture.

Keywords—*educational Simulator, Microprocessor without Interlocked Pipeline Stages (MIPS), MIPS Assembler and Runtime Simulator (MARS), pipelined architecture, branch prediction, Pipelined Simulator and Branch Explainer (PSBE)*

I. INTRODUCTION

The advancement of technology in today’s world is happening at an extremely fast pace. Computers aids us in almost every application that humankind needs. As computers become a part of our lives, there will be more demands on Computer Engineers to continue maintaining and improving computer architecture designs. Therefore, the studies of the topic such as the computer architecture is a very important for Computer Science and Engineering undergraduate. At my University, there are courses offered to the Computer Science and Engineering students in which computer assembly, organization, and architecture are being taught to them to have the foundation on how these technologies are developed. One of the core concepts taught in Computer Architecture course is the architecture of MIPS32[1] which basically includes the instruction set, data path design, pipelining techniques, hazard detection and elimination, branch predication and much more.

Understanding the dynamics of pipelining in computer architecture, pipeline hazards, elimination of hazards and branch prediction is rather difficult if done in traditional pen and pencil method. To support learning and teaching experience and to provide students an environment to experiment, educational simulators are used. Therefore, by using simulator, undergraduates will be able to have a better and more in-depth understanding of the flow of instruction execution. There are already many existing educational tools which allows users to simulate the execution of instructions on a CPU[2, 4-9]. However, most of the currently existing educational tools are not very versatile and does not support

branch prediction which is important in pipeline processor simulation.

The main objective of the work presented in this paper is to create a new plug-in to support 5- stage pipelined simulation and branch prediction for the MIPS32 using MARS [2-4] education tool. MARS is an Integrated Development Environment (IDE) controlled by a Graphical User interface (GUI). Its features include single cycle execution of instruction with variable speed, thirty-two registers and memory visible at the same time, selection of data value display in decimal or hexadecimal, “surfing” through memory using buttons to change display to next/previous, stack location, global partition and also an integrated editor and assembler [2-4]. The proposed plug-in, called pipelined simulator and branch explainer (PSBE), will provide the existing MARS simulator with the following additional capability:

- Supports pipelined instruction execution along with hazard determination and elimination.
- Supports branch prediction technique for pipelined instruction execution.
- Display visual aids such as reservation table, which allows MARS users to better visualize and understand the execution of instructions in a pipelined processor.
- Allow users to initialise registers value before instruction execution in a pipelined instruction execution.
- Introduce versatility by allowing users to control branch and jump instruction evaluation (At which stage a branch or jump instruction is evaluated at).

MARS is an open source software, well documented on how to develop new MARS plug-in to extend the capability of MARS [2-3], therefore making it a better alternative over developing a new simulator. Section II provides comprehensive review about related educational simulators. Section III describes the proposed plug-in implementation and its functionality and user acceptance test are given in section IV and V respectively

II. RELATED WORKS

This section presents some of the well-known MIPS simulators, their strengths and weaknesses.

A. EduMIPS64[5]

EduMIPS64 simulator was created to provide students with a free and cross-platform tool to experiment with MIPS64 assembly instruction [5]. EduMIPS64 supports

floating point system and its interface was inspired from WinMIPS64[8]. EduMIPS64 simulate a 5-stage pipeline CPU with a simple datapath display and support hazard detection. However, EduMIPS64 does not have an integrated editor, and it does not support branch prediction and non-pipelined CPU simulation.

B. WebMIPS[6]

WebMIPS is a web-based 5-stage pipeline simulator that is capable of display the status of hardware in the pipeline model, as well as hazard detection and forwarding in pipeline. WebMIPS is written in ASP and can be launched the simulator web page, making it easily accessible via internet without the need to download and install any content [6]. However, the display of datapath is static. Furthermore, simulator does not support branch prediction and non-pipelined CPU simulation.

C. DrMIPS[7]

DrMIPS is open-source and licensed under the GPLv3. DrMIPS supports non-pipelined and different variant of 5-stage pipelined (with hazard detection or data forwarding or without either) CPU simulation. There are two version of DrMIPS, a PC version as well as an Android version [7]. The PC version has an interface with multiple tabs. This tab includes an integrated code editor, datapath display (which display the currently loaded CPU datapath), registers values and data memory. The code editor also contains syntax-highlighting and auto-complete. However, DrMIPS does not support branch prediction.

D. Others

Another simulator SPIM [9] only supports non-pipelined instructions. WinDLX and MIPSsim are two other MIPS simulators developed at the Vienna Institute of Technology [10]. WinDLX models the pipeline of the MIPS-like DLX architecture. It allows displaying and modifying information about CPU (pipeline, registers, I/O, memory), enabling/disabling pipeline forwarding, changing memory size. MIPSsim models the MIPS architecture as in [1], with the possibility of changing memory content, but without hazard detection and forwarding units in the pipeline. Table I, presents a summary of comparison for the most important simulators [5-7] based on their capability to do pipelining and branch prediction.

TABLE I. COMPARISON OF MIPS SIMULATORS [5-7]

Capability	EduMIPS64[5]	WebMIPS[6]	DrMIPS[7]
Open Source	Yes	Yes	Yes
Code editor	No	Yes	Yes
Singlecycle (non-pipelined)	No	No	Yes
Pipeline	Yes	Yes	Yes
Visual Datapath	Simple	Static	Detailed
Branch Prediction	No	No	No

From Table I, it is apparent that the existing simulators mentioned above does not support Branch Prediction technique for a pipelined processor instruction execution.

III. PROPOSED PSBE PLUG-IN IMPLEMENTATION

To develop the proposed PSBE plug-in, the **mars.tools** package in the MARS source code folder is used. This

package stores all the java file of the plug-in tool, whereby there is a java file named "IntroToTools.java" which contain comments and detail on how to create a new plug-in tool such as what method must be implemented or what method. More details of how to develop a MARS plug-in can be found in [2-3]. The PSBE plug-in will have a five-stage pipeline architecture with each of the stage implemented as a class. The five stages are: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory (MEM), Write Back (WB).

A. Pipelined Instruction Implementation

Since the original version of MARS simulator only support unicycle/ non-pipelined instruction, a separate set of pipelined instruction will be needed to ensure the correctness of instruction execution. To create a new set of pipelined instruction, a new "simulatePipeline" method is added in SimulationCode.java interface class. The files are found in the **mars.mips.instructions** package.

MIPS 32 [1] processor has four distinct types of instruction formats depending on the maximum number of operands explicitly specified in instructions, the use of registers and memory and control flow. They are Register_FORMAT, Immediate_BRANCH_FORMAT, Immediate_FORMAT and Jump_FORMAT, as listed in Table II. The details of the instructions given in Table II is available in [1-4]. The datapath of the MIPS 32[1] 5-stage pipeline is shown in detail in Fig. 1.

TABLE II. LIST OF ALL FOUR FORMAT PIPELINED INSTRUCTION

Instruction Type	Instruction
Register FORMAT	nop, add, sub, addu, subu, slt, nor, xor, sll, sllv, jr, srl, sra, mul, mult, multu, div, divu, mfhi, mflo, mthi, mtlo, and, or, srlv, srav
Immediate FORMAT	addi, addiu, andi, ori, xori, lui, lw, sw, lb, sb
Immediate BRANCH_FORMAT	beq, bne, bgez, bgtz, blez, bltz
Jump_FORMAT	j, jal

B. Data and Control hazards

With unicycle instruction execution, each instruction is fully executed before any other instruction is executed. Thus there is no dependency issues for the next instruction with earlier instruction. However, with the implementation of a pipelined processor, each instruction is separated into 5-stages according to the pipeline architecture design shown in Fig.1. Therefore, each instruction may not be fully executed before the subsequent instruction is being executed. Because of this, data hazards and control hazards [1] occur in pipelined architecture.

To detect the presence of data and control hazards, manual hazard checker is created in the proposed PSBE plug-in. The manual hazard checker checks for any data and control hazards in the MIPS code assembled by the user and corrects the MIPS code if hazards are presents, it then displays a pop-up message that contain the corrected MIPS code. The

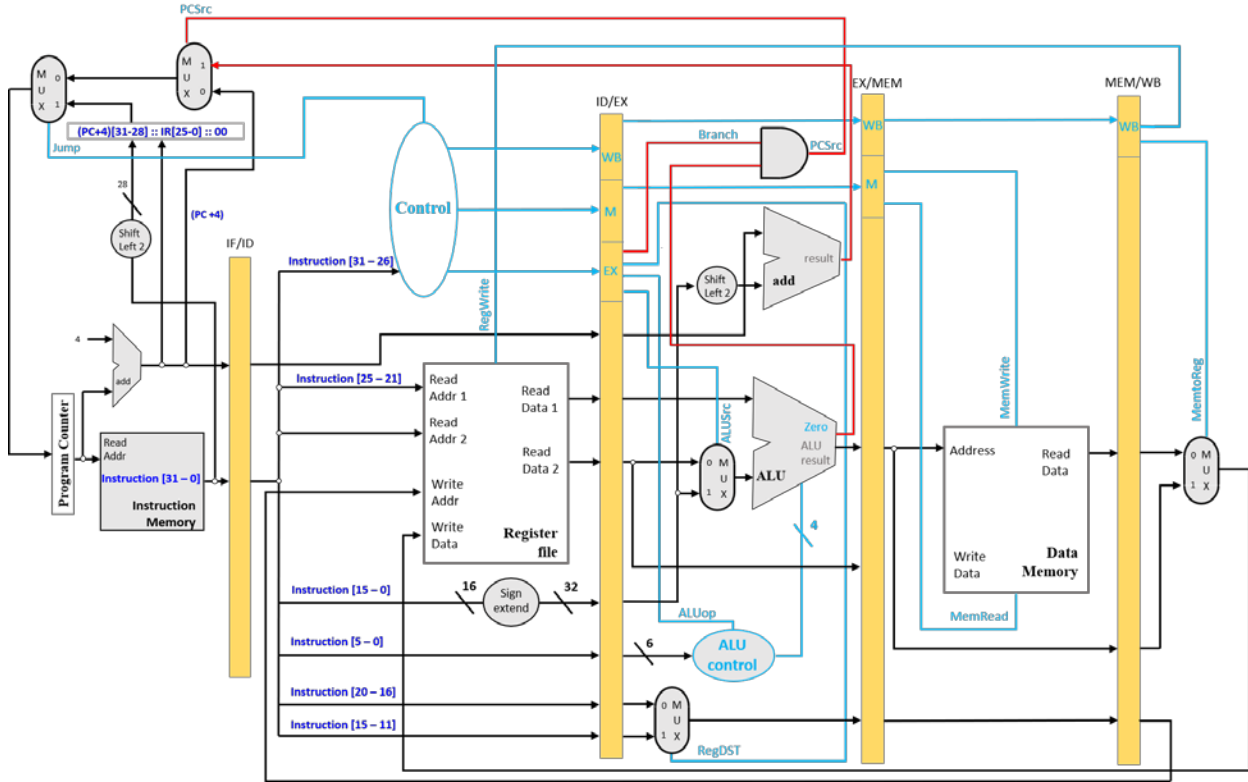


Fig 1. 5-stage MIPS pipeline datapath implemented in the plug in. (default case- jump is resolved in the ID stage and branch in the EXE stage)

correction of MIPS code is done by adding “Nop” instruction to the Original MIPS code. A “Nop” instruction is a meaningless instructions which only helps in wasting a clock cycle. Simulators [5-7] implements automatic hazard detection and correction. This method may create confusion to students to understand the hazard detection and elimination correctly. In the proposed PSBE plug-in, the user can learn by comparing the original MIPS code with the corrected MIPS code and identify the dependency and correct them.

To overcome data hazards, technique as such Data Forwarding is implemented to reduce the number of stall cycles needed. To overcome Control hazards, flexible implementation of hardware as well as the Branch Prediction technique is implemented. Flexible implementation of where branch and jump are resolved is explained in section III C. Two type of branch prediction are implemented in the proposed PSBE plug-in, Static and Dynamic branch prediction. More detail can be found in section III part D.

C. Branch and Jump Instruction Control

In a pipelined processor, branch and jump instruction can be evaluated at different clock cycles/stages. And with evaluation of Branch and Jump instruction done at different stages, it can have different impact (control hazards) on the flow of the instructions’ execution, which in turn can affect the results correctness. The proposed plug-in allows the user to control at which stages a branch or jump instruction is being evaluated at. This is a flexibility which is not offered in any of the simulators discussed in literature.

For Branch instruction, the evaluation can occur at either of these stages:

- ID stage

- EX stage (default)
- MEM stage

For Jump instruction, the evaluation can occur at either of these stages:

- ID stage (default)
- EX stage

With evaluation at different stages, the underlining hardware structure will defer accordingly to the different configuration set by the user.

D. Branch Prediction Technique Implementation

Branch prediction in the proposed plug-in is implemented such that it always evaluates a branch instruction at the ID stage, meaning that all branch instruction will update the Program Counter (PC) at ID stage according to the prediction. However, if branch prediction is used and the user choose to evaluate a branch instruction at the ID stage (using the Branch and Jump Instruction Control), the PC will be updated according to the actual evaluated result instead of according to the prediction.

There are four possible case for the prediction and actual result when using the branch prediction. The four possible case are:

1. Prediction: Taken → Actual: Taken
2. Prediction: Taken → Actual: Not Taken
3. Prediction: Not Taken → Actual: Taken
4. Prediction: Not Taken → Actual: Not Taken

Two Type of Branch prediction scheme, static prediction and dynamic prediction is implemented. For both type of prediction scheme, the user is required to set an initial prediction for branch instruction, the prediction can either be 'Taken' or 'Not Taken'. However, in static prediction scheme, the initial prediction set by the user will not change throughout the entire MIPS code execution. For dynamic prediction scheme, the initial prediction set by the user may change throughout the entire MIPS code execution. Out of many types of dynamic prediction schemes, two types of dynamic prediction scheme, 1-bit prediction and 2-bit prediction are implemented in the proposed plug-in. For 1-bit prediction, the user can set the initial prediction to either State 0 (Not Taken) or State 1 (Taken). The prediction changes throughout the MIPS code execution according to the transition diagram shown in Fig. 2.

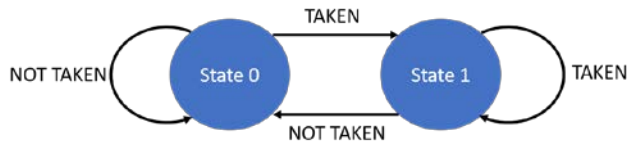


Fig. 2. 1-bit Prediction Scheme Prediction State Transition Diagram

For 2-bit prediction, the user can set the initial prediction to either State 0 (Not Taken), State 1 (Not Taken), State 2 (Taken) or State 3 (Taken). The prediction changes throughout the MIPS code execution according to the transition diagram shown in Fig. 3.



Fig. 3. 2-bit Prediction Scheme Prediction State Transition Diagram

E. Settings Interface

The control and data hazard countermeasure, branch and jump instruction control, and branch prediction technique as mentioned is the part B, C, and D of section III respectively

are part of the settings that the users can control. A screenshot of settings interface with default settings is shown in Fig. 4.

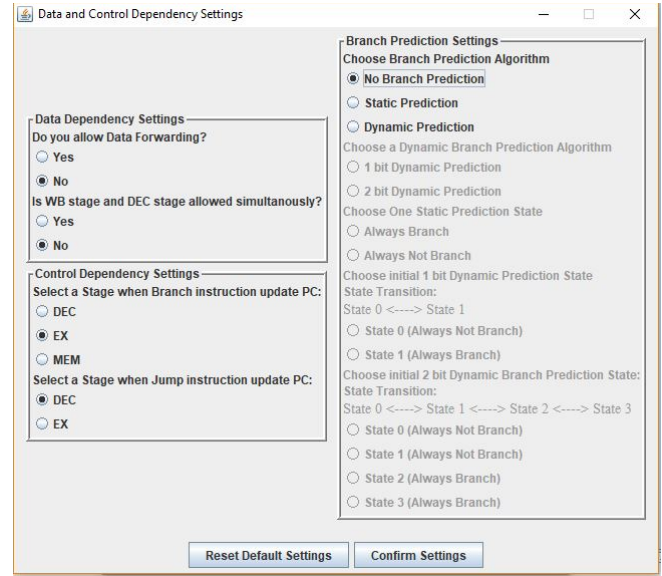


Fig. 4. Screenshot of the plug-in Settings Interface with Default settings

F. Visual Aids

For visual aids to represent pipeline simulation, the author decided to use a reservation table as it provide a simple yet easily explainable and understandable visual for user. When branch prediction scheme is used, the proposed plug-in will also display the branch decision path, as such indicate at which state the decision was made, the final state (for dynamic prediction only) and, the CPI (cycle per instruction) and Steady State CPI. Fig. 5 shows a screenshot of the visual aids used in the proposed plug-in.

IV. FUNCTIONALITY TEST

To verify that the purposed plug-in is functioning correctly, a test bench MIPS code as shown in Fig. 6, that covers all the feature functionality was used.

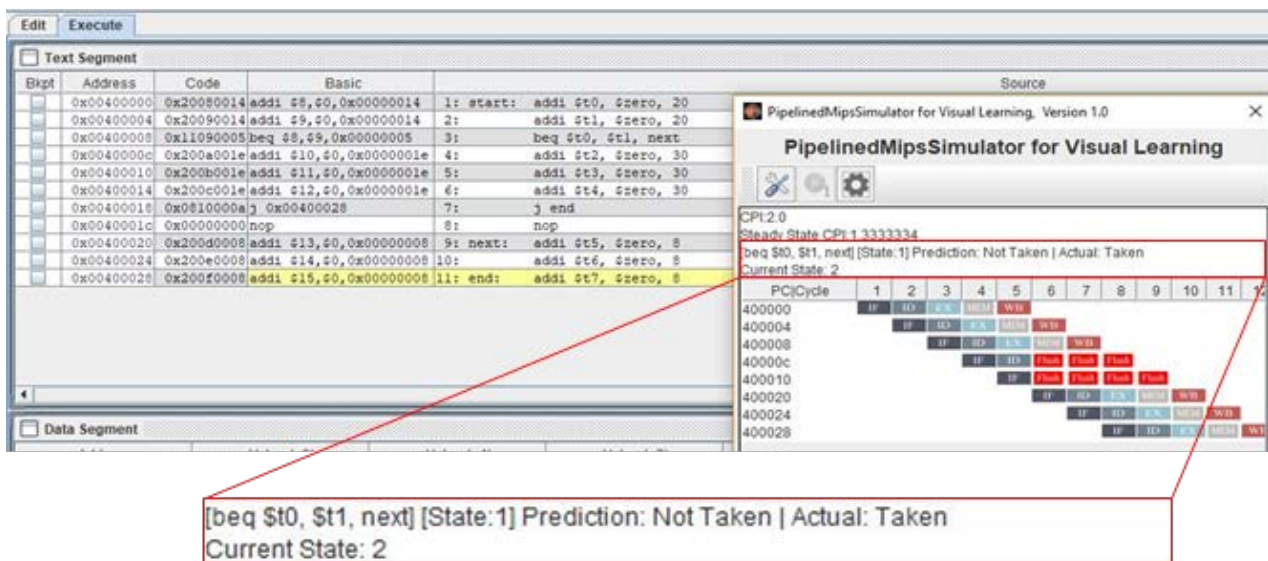


Fig. 5. Reservation Table visual aids with additional branch decision information, CPI and steady state CPI

```

start:  addi $t0, $zero, 20
        addi $t1, $zero, 20
        bne $t0, $t1, next
        addi $t2, $zero, 30
        addi $t3, $zero, 30
        addi $t4, $zero, 30
        beq $t3, $t4, end
next:   addi $t5, $zero, 8
        addi $t6, $zero, 8
        addi $t7, $zero, 8
end:    addi $s0, $zero, 8

```

Fig. 6. Test Bench MIPS code used for functionality test

A. Testing correctness of Manual Hazard Checker

As mentioned in Section III part D, the Manual Hazard Checker checks for any data and control hazards and then correct them accordingly by adding 'Nop' instruction in between the instruction with dependency. However, the number of 'Nop' instruction inserted by the checker varies depending on the data and control dependency settings that is set by the user as shown in Fig. 4. Table III and IV shows the number of 'Nop' should be added to ensure correctness of results for Data hazard and Control hazard respectively.

TABLE III. Data Hazard Correction based on Countermeasure used

Countermeasure Used	Correction
Full Data Forwarding	No correction
Simultaneous Instruction Decoding and Write Back	2 Nop Instruction
No Countermeasure (default architecture as in Fig. 1)	3 Nop instruction

TABLE IV. Control Hazard Correction based on Countermeasure used

Countermeasure Used	Correction
No Branch Prediction (Jump or Branch instruction evaluated at ID)	1 Nop Instruction
No Branch Prediction (Jump or Branch instruction evaluated at EX)	2 Nop Instruction
No Branch Prediction (Branch instruction evaluated at MEM)	3 Nop instruction
Static Prediction or Dynamic Prediction	No correction

Using the default settings (Data Forwarding is Not Enabled, Decode and WriteBack cannot occur simultaneously, Branch evaluated at Execute Stage, Jump evaluated at Decode Stage and No Branch Prediction technique is being used) with the test bench MIPS code, the number of 'Nop' instruction added is consistent with the values presented in Table III and Table IV as given in Fig. 7. With the setting of data forwarding enabled and using 2-bit dynamic prediction as shown in Fig. 8(a), the corrected MIPS code generated by the manual hazard checker is shown in Fig. 8(b).

```

New corrected MIPS program Codes

start:      addi $t0, $zero, 20
            addi $t1, $zero, 20
            nop #nop added by checker
            nop #nop added by checker
            nop #nop added by checker
            bne $t0, $t1, next
            nop #nop added by checker
            nop #nop added by checker
            addi $t2, $zero, 30
            addi $t3, $zero, 30
            addi $t4, $zero, 30
            nop #nop added by checker
            nop #nop added by checker
            nop #nop added by checker
            beq $t3, $t4, end
            nop #nop added by checker
            nop #nop added by checker
next:       addi $t5, $zero, 8
            addi $t6, $zero, 8
            addi $t7, $zero, 8
end:        addi $s0, $zero, 8

```

Fig. 7. Corrected MIPS code generated by the manual hazard checker using default setting (jump evaluated at ID and branch at MEM stage)

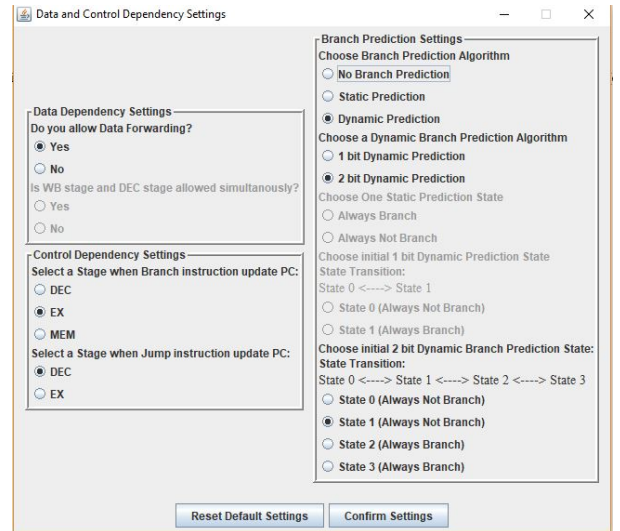


Fig. 8(a) Screenshot of the plug-in Settings Interface with dataforwarding and 2-bit dynamic predictor

```

New corrected MIPS program Codes

start:      addi $t0, $zero, 20
            addi $t1, $zero, 20
            bne $t0, $t1, next
            addi $t2, $zero, 30
            addi $t3, $zero, 30
            addi $t4, $zero, 30
            beq $t3, $t4, end
next:       addi $t5, $zero, 8
            addi $t6, $zero, 8
            addi $t7, $zero, 8
end:        addi $s0, $zero, 8

```

Fig. 8(b). Corrected MIPS code generated by the manual hazard checker using data forwarding and 2-bit dynamic prediction

B. Testing correctness of Pipelined processor simulation

As per the settings shown in Fig 8(a), the instruction “addi \$t5, \$zero, 8” and “addi \$t6, \$zero, 8” are flush from the pipelined as expected. The register value of \$t5 and \$t6 remain zero as the instruction are flushed and not execution fully. The final registers value is shown in Fig. 9.

Registers	Coproc 1	Coproc 0
Name	Number	Value
\$zero	0	0x00000000
\$at	1	0x00000000
\$v0	2	0x00000000
\$v1	3	0x00000000
\$a0	4	0x00000000
\$a1	5	0x00000000
\$a2	6	0x00000000
\$a3	7	0x00000000
\$t0	8	0x00000014
\$t1	9	0x00000014
\$t2	10	0x0000001e
\$t3	11	0x0000001e
\$t4	12	0x0000001e
\$t5	13	0x00000000
\$t6	14	0x00000000
\$t7	15	0x00000000
\$s0	16	0x00000008
\$s1	17	0x00000000
\$s2	18	0x00000000
\$s3	19	0x00000000
\$s4	20	0x00000000
\$s5	21	0x00000000
\$s6	22	0x00000000
\$s7	23	0x00000000
\$t8	24	0x00000000
\$t9	25	0x00000000
\$k0	26	0x00000000
\$k1	27	0x00000000
\$gp	28	0x10008000
\$sp	29	0x7ffffcfc
\$fp	30	0x00000000
\$ra	31	0x00000000
pc		0x0040002c
hi		0x00000000
lo		0x00000000

Fig. 9. Register Values updated

The result after the execution of the test bench MIPS code are shown in Fig. 10.

V. USER ACCEPTANCE TEST

After the completion of the proposed PSBE plug-in, a user acceptance test was done by testing the PSBE plug-in against others existing simulator such as DrMIPS, WebMIPS, and EduMIPS64. Ten students of age group 21 to 24 years from my university who have completed Computer Architecture modules were selected to do the user acceptance test.

TABLE V. USER ACCEPTANCE TEST

Question	Total Score			
	PSBE Plug-in	Dr. MIPS [7]	EM [5]	WM [6]
Please rate how user-friendly (easy and intuitive) are the educational simulators (from 1 - 10 with 1 being non easy and unintuitive and 10 being easy and very intuitive).	8.2	7.4	5.6	4.8
Please rate how useful is the simulators in understanding data and control dependency (from 1 - 10 with 1 being difficult and 10 being easily understandable).	8.8	7	4	4
Please rate how flexible is the simulators to modify the settings for data and control hazard detection (from 1 - 10 with 1 being non-flexible and 10 being extremely flexible).	9.2	7.6	3	3.2
Please rate how helpful are the visual aids used in the educational simulators (from 1 - 10 with 1 being very unhelpful and 10 being very helpful).	8.6	7.8	6	4.8
Please rate how likely you would use the educational simulators again (from 1 - 10 with 1 being No thanks and 10 being absolutely).	8.6	7.6	5.6	4
Total Score	43.4	37.4	24.2	20.8

The students were given a zip file containing the simulators DrMIPS, proposed PSBE simulator and EduMIPS64 with a text file containing a URL link pointing to the WebMIPS. There are also four different assemble file (.asm) each containing the same instruction set example with each file catering to different simulator due to a slight different in syntax that each simulator is able to handle. Each student is given enough information to allow them to assemble and run the assembly code. No time limit was imposed testing of each of the individual simulation. Once the assembly code have fully executed and the result are displayed based on the visual aids used by each simulator, the student are given the URL link to the online questionnaire to evaluate each simulators. The survey question and results are shown in Table V. In Table V, WM stands for WebMIPS and EM stands for EduMIPS64.

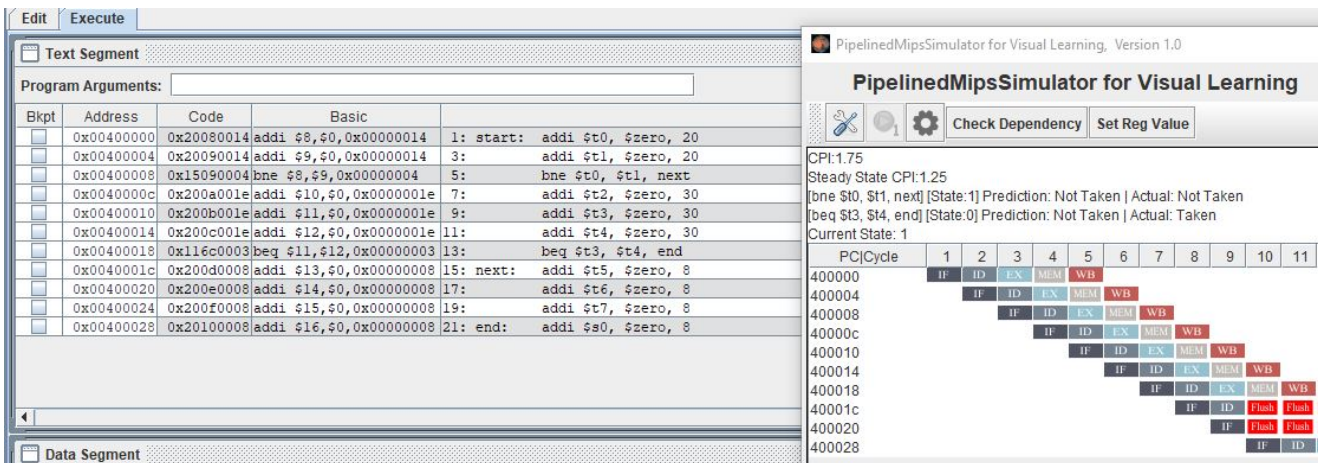


Fig. 10. Pipelined processor simulation with data forwarding enable and 2-bit dynamic branch prediction

As a basic assessment about the usefulness of the simulator, students were required to answer a feedback questionnaire. The questionnaire is an online questionnaire created on the survey monkey website and the questions are given in table V. The questions that had to be answered are on a Likert scale from 1-10, where 10 is the best case value and 1 would be the worst case value. The questionnaire compares the user friendliness, usefulness in terms of understanding data and control hazard, flexibility, helpfulness in terms of visual aids and the likelihood to reuse the simulator tools. The maximum score that any user can provide is now capped at 50 and the proposed PSBE simulator scored 43.4 out of 50 (86% acceptance) when compared to other simulators in literature as can be noted from Table V. The users appreciated the usefulness of the pipelined simulator to understand the data and control dependency and how to do the countermeasure in the proposed PSBE plug in. The high scores of proposed PSBE simulator indicates that the users also liked the flexibility in the settings to understand in depth, the effects (in terms of number of stall cycles) of branch and jump been evaluated at different stages. The proposed PSBE plug-in enables better understanding of pipelined architecture, data and control hazard detection and elimination and branch prediction concepts.

VI. CONCLUSION AND FUTURE DEVELOPMENT

The proposed PSBE plug-in provides the MARS simulator with the capability to support pipelined instruction execution by understanding control and data hazards, as well as allows user to understand branch prediction technique in detail. With basic visual Aids like reservation table, PSBE plug-in is suitable for undergraduate students with little knowledge and experience with computer architecture. The proposed plug-in (.jar file) is made available at <https://www.ntu.edu.sg/home/smitha>. However, further improvement to the current visual aids can be made to provide more in-depth details and information to the MARS user, such as displaying the underlying datapath for the

pipelined processor simulation as similar to MARS X-ray [11] for single cycle architecture.

ACKNOWLEDGMENT

The authors would like to thank Nanyang Technological University, School of Computer Science and Engineering for their support.

REFERENCES

- [1] D. A. Patterson and J. L. Hennessy, *Computer Organization and Design MIPS Design - The Hardware/Software Interface*, 5th ed. Morgan Kaufmann, 2013.
- [2] K. Vollmar, "MARS MIPS simulator - Missouri State University," [Online]. Available: http://courses.missouristate.edu/KenVollmar/mars/CCSC-CPmaterial/MARS_tutorial.doc.
- [3] K. Vollmar, "MARS 4.5 help contents," [Online]. Available: <http://courses.missouristate.edu/KenVollmar/mars/Help/MarsHelpTols.html>.
- [4] K. Vollmar and P. Sanderson, "MARS: an education-oriented MIPS assembly language simulator," *SIGCSE Bull.*, vol. 38, no. 1, pp. 239-243, 2006.
- [5] "EduMIPS64," [Online]. Available: <https://www.edumips.org/>.
- [6] I. Branović, R. Giorgi and E. Martinelli, "WebMIPS: a new web-based MIPS simulation environment for computer architecture education," *WCAE '04 Proceedings of the 2004 workshop on Computer architecture education*, June 2004.
- [7] B. Nova, J. C. Ferreira and A. Araujo, "Tool to support computer architecture teaching and learning," in *2013 1st International Conference of the Portuguese Society for Engineering Education (CISPEE)*, Porto, 2013.
- [8] "WinMIPS64," Available: <http://indigo.ie/~mscott/>
- [9] J. Larus, "SPIM MIPS Simulator," [Online]. Available: <http://spimsimulator.sourceforge.net/>.
- [10] H. Grunbacher, "Teaching computer architecture/organisation using simulators", *FIE '98. 28th Annual Frontiers in Education Conference. Moving from 'Teacher-Centered' to 'Learner-Centered' Education.*, pp: 1107-1112, 1998.
- [11] Guilherme C. R., R. D. Araújo, Flávio L. C. Pádua, Fábio L. Corrêa, "MIPS X-Ray: A plug-in to MARS simulator for datapath visualization", *2nd International Conference on Education Technology and Computer*, pp: 32- 36, 2010.